第6讲：测后重点补强

ICS 测后补强 · 发布版 · 2026-10-13 · 知识讲解与三栏命令表

1 精度题：每一步算得准，才能放心移动括号
2 访问次数：先画出数据去了哪里，再数
3 movz与movs：新增加的高位到底填什么
4 条件传送：准备两个候选，再根据标志选一个
5 ABI寄存器保护：谁用坏了，谁负责哪一份
6 变量放寄存器还是内存：先问程序要不要它的地址
7 数组与指针：先确定从哪里开始、一步多大
8 结构体：成员不能随便紧挨着，整体也要补齐
9 浮点机器代码：先分清地址、整数值和浮点值
10 联合、字节序与防护：从具体内存格子出发
11 汇编命令速查表：命令、含义、读题要点
12 做题时固定问自己的六句话
13 来源与整理说明

本讲根据你完成2025第一次阶段测验后的反馈编写。这里的“第6讲”是**本套复习资料新增的一讲**；试卷中“第3讲、第4讲”等指原课程讲次。题号只用来定位薄弱点，下面用新例子重新讲知识与读题方法，不重复抄整卷答案。

适用前提：x86-64 / Linux / System V AMD64 / AT&T语法；char、short、int、long、指针分别占1、2、4、8、8字节。浮点使用IEEE binary32、binary64和最近偶数舍入。题目明确另给字节序或模型时，以题面为准。

建议先读第2至6节，把指令与调用弄清；再读第1节和第7至10节。第11节保留此前速查表的89条顺序，另补本次需要的6种具体写法。无需一次背完：每遇到指令，能说出“读什么、做什么、写哪里、改不改标志”即可。

你的反馈	本讲对应位置	学完后应能做什么
原课程第3讲，试卷第6题B：精度	第1节	用中间结果范围判断是否需要舍入
原课程第4讲，试卷第8题：访问次数	第2节	分别列寄存器读写、数据内存读写
指令不熟，尤其第11题的扩展	第3节、第11节	拆出动作、源宽度、目标宽度
第12题第4问：条件传送	第4节	检查标志、候选值与提前求值风险
ABI中的寄存器保护	第5节	说清谁负责保存、什么时候需要保存
原课程第6讲，试卷第17题：变量位置	第6节	区分可放寄存器、需要地址、跨调用保存
原课程第7讲，试卷第20题：数组表达式	第7节	区分加一项、加一行、取地址、取内容
第21至24题	第8至10节及第5节	推对齐、读浮点传参、读联合字节、分清防护

1 精度题：每一步算得准，才能放心移动括号

1.1 范围和精度问的是两件事

想象一把刻度尺：**量程**回答“最大能量到哪里”，**刻度间距**回答“两个很接近的数能否区分”。浮点数也如此。float能表示数量级很大的数，却不一定能区分两个相邻大整数。

类型	有效二进制位数（规格化数）	连续整数精确表示的保证范围
float / binary32	24位，含隐藏的1	-2^{24} 至 2^{24} 的每个整数
double / binary64	53位，含隐藏的1	-2^{53} 至 2^{53} 的每个整数

超过这段范围，不代表“任何数都存不下”：例如2^{60}仍能被double精确表示，因为它只有一个非零二进制位；但$2^{60}+1$不能。区别在于低位是否必须保留。

1.2 先用一个float例子看见“丢失低位”

在$[2^{24}, 2^{25})$中，float相邻刻度相差2。附近有16777216、16777218、16777220，却没有16777217。

```
可表示：16777216 ——— 16777218 ——— 16777220
精确数：                    ↑ 16777219 在后两者正中间
```

中点两边距离同为1，用最近偶数规则：把候选除以刻度单位2，得到8388609（奇）和8388610（偶），因此选16777220。“偶”指最低保留二进制位为0，不是看候选十进制整数是否能被2整除；这里两个候选本来都是偶数。

一般规格化正数位于$[2^E, 2^{E+1})$时，若有效精度为p位，刻度间距为$2^{E-(p-1)}$。不用先背推导：小数部分只保留p−1位，末位权重就是该间距。二次幂边界的左右间距可能不同，找邻居时要看落在哪一段。

补充问答：Q031：刻度间距为什么随数值变大？用4位精度逐项看。

1.3 加法结合律题，先检查输入限制

假设x、y、z都是32位int，先分别转成double，再比较两种加法顺序。每个整数绝对值不超过2^{31}；两个数的数学和绝对值不超过2^{32}，三个数不超过$3\times 2^{31} < 2^{33}$。

这几个中间结果**仍然是整数**，并且远在double的连续整数精确范围$[-2^{53}, 2^{53}]$内。因此每一步都精确，两种括号顺序得到同一个数学和。

这里“浮点一般不满足结合律”不能替代具体分析。输入是任意double时可能失败；输入限制为这种小整数且中间加法都精确时，结合律成立。

补充问答：Q032：只改变括号，double加法怎样分别得到0和1？ 。

读题步骤： 先看变量来源 → 判断运算类型 → 给每个中间结果估上界 → 检查有效精度 → 再判断等式。

1.4 为什么加法够用，乘法就不一定够用

两个绝对值约2^{31}的整数相乘，乘积数量级可到2^{62}。这已经超过double对所有整数都精确的保证范围。它不说明每次乘法都舍入，但说明不能再证明“全部输入都精确”。三个数连乘时，中间舍入会影响后一乘。

用float新例子看分组：令a=2^{25}、b=-2^{25}、c=1，三个输入都能精确存入float。

- `(a+b)+c`：先算a+b=0，再加1，得到1。
- `a+(b+c)`：精确的b+1是−33554431，邻近刻度是−33554432与−33554430，恰好在正中间。按最低保留位取偶数，舍入回−33554432，也就是b；再加a，得到0。

两种路径的区别来自**先算b+c时发生了舍入**。不要只说“数很大所以加1一定没用”：二次幂边界两侧刻度间距不同，必须先找出实际邻居。

1.5 还要看转换发生在运算前还是后

下面假设x、y均为32位int。

```
double a = (double)x + y;   // x显式转double，y自动转double，再相加
double b = (double)(x+y);   // 先在int中加；若溢出，后转double救不回来
```

第一行的(double)只直接作用于x；但加号两边出现double与int时，通常算术转换会把y的值也转为double，所以做的是浮点加法。转换不改变原变量x、y的int类型。第二行先算括号中的int加法，再转换结果；左侧接收变量是double，并不能改变右侧加法的类型。

补充问答：Q033：只写了(double)x，为什么y也会转成double？。

在课程C规则下，有符号int加法溢出是未定义行为，不能把它当作已经算对的整数再转double。另两条常考规则：有限float转double能保留原值，但不能补回此前舍入丢失的位；浮点转int向零截断，还必须检查截断后的整数能否由目标int表示。

2 访问次数：先画出数据去了哪里，再数

2.1 地址也放在寄存器里，但读取地址不等于读取内存

设rdi=0x1000，内存0x1000起的8字节表示42：

```
rdi这个寄存器：0x1000 ──用作地址──→ 内存[0x1000..0x1007]：42
```

指令	实际动作	通用寄存器读/写	数据内存读/写
`movq %rdi,%rax`	把0x1000复制到rax	读rdi、写rax：2次	0次
`movq (%rdi),%rax`	到0x1000处取42，放rax	读rdi、写rax：2次	读8字节：1次
`movq %rax,(%rdi)`	把rax的值写到0x1000处	读rax、读rdi：2次	写8字节：1次
`leaq 8(%rdi),%rax`	计算0x1008，放rax	读rdi、写rax：2次	0次

最后一行有括号但没有读内存，因为lea只计算地址式。CPU需要先读取rdi里的地址数值，这就是一次寄存器读取，不因此多出一次数据内存访问。

2.2 “一次”的计数口径

本讲按试卷的教学口径，数**体系结构层面的数据读写动作**：一次读和一次写分别计数；movq连续读取8字节仍算一次读取。题目没有要求时，不计取指、不计RIP和条件码寄存器、不推算缓存或真实电路次数。

指令	通用寄存器次数	数据内存次数	原因
`movq $7,%rax`	写rax：1	0	7是指令里的常量，不是另一次数据内存读取
`addq %rbx,%rax`	读rbx、读旧rax、写新rax：3	0	目的既是输入也是输出
`addq (%rdi),%rax`	读rdi、读rax、写rax：3	读：1	内存提供一个加数
`addq %rax,(%rdi)`	读rax、读rdi：2	读旧值、写新值：2	内存目的要先读后写
`cmpq %rbx,%rax`	读rbx、读rax：2	0	只写标志，不写回rax
`leaq 8(%rdi,%rsi,4),%rax`	读rdi、读rsi、写rax：3	0	计算rdi+4×rsi+8

不要数寄存器名字有几个： addq的rax虽只出现一次，却要读旧值再写新值。push/pop/call/ret还有隐含的rsp和栈内存动作，也不能只看显式操作数。若题目另规定“访问过几个不同寄存器”或“几个内存操作数”，那是另一种计数，不套这张表。

3 movz与movs：新增加的高位到底填什么

3.1 同一字节，两种扩展得到不同数

设al存的是`1111 1101`，十六进制FD。作为8位无符号数它是253；作为8位补码有符号数它是−3。

```
零扩展到32位：00000000 00000000 00000000 11111101 = 253
符号扩展到32位：11111111 11111111 11111111 11111101 = −3（32位补码）
```

零扩展补0，是为了保留原无符号数值；符号扩展复制**源的最高位**，是为了保留原补码有符号数值。CPU并不知道C变量叫什么类型，是编译器选出的指令决定如何补位。若源最高位为0，两种扩展结果一样。

3.2 把名字拆开读

写法	拆分	通俗含义
`movzbl %al,%eax`	mov + z + b + l	复制al的8位，补0扩成32位，放eax
`movsbl %al,%eax`	mov + s + b + l	复制al的8位，按al最高位扩成32位
`movsbq %al,%rax`	mov + s + b + q	8位直接符号扩展到64位
`movslq %eax,%rax`	mov + s + l + q	32位符号扩展到64位
`movl %eax,%edx`	mov + l	复制32位；因写edx，rdx高32位自动清0

b/w/l/q分别是8/16/32/64位。movz、movs在这里是指令族简称，不是完整的固定宽度指令。普通32→64零扩展用写32位寄存器完成，不要硬造`movzlq`。

GNU汇编手册按“扩展方式＋源宽度＋目标宽度”解释这种命名：AT&T扩展传送命名。

3.3 两层规则别混成一层

仍设al=FD，分别独立执行：

操作	eax或rax直接得到的位模式	整个rax的最终位模式
`movzbl %al,%eax`	eax=000000FD	00000000000000FD
`movsbl %al,%eax`	eax=FFFFFFFD	00000000FFFFFFFD
`movsbq %al,%rax`	rax=FFFFFFFFFFFFFFFD	FFFFFFFFFFFFFFFD

第二行把8位符号扩展成32位后，**写eax仍清rax高32位**。所以它保留了32位−3，却不能把整个rax直接读成64位−3。想得到64位−3，可直接movsbq，或在movsbl后再movslq。

3.4 setcc后为什么常跟movzbl

假设eax原来是`0x12345678`。执行`sete %al`且条件成立后，只把最低字节改为01，所以eax是`0x12345601`，不是整个整数1。

随后`movzbl %al,%eax`把这个01补0扩成`0x00000001`，才能作为完整int布尔值使用。若条件为假，扩展后得到0。

setcc只会产生0或1，它们符号位都是0，所以这个特例中movsbl也会得到同样32位结果；**两种指令效果在0/1上相同，不意味着它们普遍等价。** 用FD这类最高位为1的字节才能检验区别。

4 条件传送：准备两个候选，再根据标志选一个

4.1 cmov需要准备什么

普通整数cmov的源是寄存器或内存，目的必须是寄存器；没有8位形式，也不能直接把立即数当源。它根据已有条件码决定要不要复制，**不自己比较两个操作数的大小**。

必须准备的东西	作用
目的寄存器里已有的默认值	条件不成立时，用这个候选
源操作数里的另一候选值	条件成立时，用它替换默认值
正确的条件码	由cmp、test或相应运算建立，用来判断条件
匹配的条件后缀和宽度	signed用l/g等；unsigned用b/a等；数据宽度另判断

以有符号long中较小的值为例，x在rdi，y在rsi：

```
movq  %rdi, %rax    # 默认答案为x
cmpq  %rsi, %rdi    # 用x-y设置标志，比较x和y
cmovg %rsi, %rax    # 若x>y，说明y更小，就改用y
ret                 # 返回rax中的答案
```

输入	默认rax	x>y是否成立	最终rax
x=9，y=4	9	是	复制y，得4
x=2，y=7	2	否	保留2

`cmovg`的g是greater。`cmovl`的l是less；不能把它当成movl的32位宽度后缀。条件传送的数据宽度可由寄存器看出。

4.2 为什么中间可能要补一条cmp

```
cmpq %rsi, %rdi     # 标志描述x-y
subq %rcx, %rdx     # 标志被覆盖，现在描述rdx-rcx
cmovg %rsi, %rax    # 使用的是sub留下的标志!
```

如果要按x>y选择，应在候选计算结束后重新cmp。mov、lea、setcc、jcc、cmovcc不改这些整数标志；add、sub和逻辑运算会改。inc/dec只保留CF，会改变ZF、SF、OF，因此也可能破坏有符号比较条件。

4.3 什么时候不适合用cmov

条件传送没有让控制流程绕过一段代码。若你为了给它准备候选，已经做了昂贵计算、除零、无效地址读取、函数调用或其他副作用，最后“不选择”该候选也不能撤销已经发生的事。

```
return p != 0 ? *p : 0;
```

不能先无条件读*p，再靠cmov筛掉无效结果。传统cmov以内存为源时，即使条件不成立，也不能依赖它避免该内存读取或异常。Intel指令手册的CMOVcc说明

做题时检查：两侧值是否可安全提前算？是否浪费大量工作？比较标志有没有被覆盖？条件是否有符号？另一个细节：32位cmov即使条件不成立，目的64位寄存器的高32位仍清零；“保持目的不变”指保留目标32位值。上面的64位例子没有这个高位问题。

5 ABI寄存器保护：谁用坏了，谁负责哪一份

5.1 先明确调用关系

A执行`call B`，A是调用者，B是被调用者。若B再调用C，那么B相对于C又是调用者。这两个名字是当前关系，不是函数终身身份。

类别	System V AMD64寄存器	谁负责旧值	什么时候要保存
调用者保存	rax、rcx、rdx、rsi、rdi、r8、r9、r10、r11	A若还要旧值，由A保存	该旧值在B返回后仍要用，且不能依赖B保留时
被调用者保存	rbx、rbp、r12、r13、r14、r15	B若改动，由B恢复	B要覆盖这些寄存器时；未改动无需备份
栈指针	rsp	B必须恢复正常返回所需的栈状态	ret前指向自己的返回地址；ret后回到A调用前的位置
常用浮点寄存器	xmm0至xmm15	都不保证跨调用保留旧内容	A有浮点旧值还要用时，需要另行保存或重算

“保存”可以用栈，也可以转存到合适位置；不要求机械地push全部寄存器。call只保存返回地址，ret只取返回地址，**它们不自动备份、恢复所有通用寄存器**。这里采用Linux/System V规则，不能混用Windows x64的寄存器表。规范依据：AMD64 psABI寄存器与调用约定。

5.2 一个同时出现两种责任的完整例子

A收到参数x，还要调用B得到另一个值，并返回x+B()。假定B不需要参数，x和结果都用long。

```
A:
    pushq %rbx          # A要改rbx，先保存“A的调用者”的旧rbx
    movq  %rdi,%rbx     # x原在rdi，转存到跨B调用可保留的rbx
    call  B             # B可改rdi、rax；但有义务保留进入B时的rbx
    addq  %rbx,%rax     # B的返回值 + 保存的x
    popq  %rbx          # 还原“A的调用者”的旧rbx
    ret
```

这里有两份不同的“旧值”：push保存的是A入口时的旧rbx；mov保存的是当前x。B如果自己使用rbx，也要备份并恢复它在B入口时的内容，也就是x。

这个单次push还让常见入口对齐状态满足下一次call：A入口rsp模16为8，push后模16为0，执行call B前满足16字节对齐。不能随便多push一项后仍假定对齐正确。

5.3 参数位置、返回位置与保护责任是三个问题

内容	简单标量参数/返回值的规则
整数、指针参数	依次rdi、rsi、rdx、rcx、r8、r9；超出可用寄存器的部分按ABI放栈
float/double值参数	独立使用xmm0至xmm7这条序列
整数或指针返回值（本讲的单个标量）	rax；int通常使用eax
float/double返回值	xmm0

例：`double f(int a, double b, long *p, float c)`中，a用edi，b用xmm0，p用rsi，c用xmm1。**指向double的指针也是地址，走整数/指针通道；只有double数值本身走浮点通道。** 复杂结构体参数另有分类规则，不用这张简表生搬。

6 变量放寄存器还是内存：先问程序要不要它的地址

6.1 三种状态都可能：寄存器、内存、被优化掉

C的变量名是源代码概念。机器代码可能把它放寄存器、放栈内存，或发现结果可直接计算而彻底省掉。一个变量还可能一段时间在寄存器、一段时间在内存。

情况	课堂读题的判断	注意边界
只用一个标量的数值，如x+1	通常可放寄存器	“可以”不等于“编译器必定这样分配”
把&x交给会读写它的外部函数	保留这个调用时，需要可寻址的内存对象	普通C指针不能指向某个通用寄存器的内部
同时活跃的值太多，寄存器不够	部分值可能暂存栈上，称溢出保存	不需要取地址，也可能用内存
一个值要跨call继续使用	需要受保护寄存器、栈保存或可靠重算	不等于必定用栈，也不能一直相信rax不变
局部数组/结构体要按地址访问	通常需要内存布局	若优化器能拆分或消除对象，实际代码可能不同
常量表达式、未被使用的变量	可能不存在独立存储	不要给每个C变量强行分配格子

6.2 用两个参数形式区分“传值”和“传地址”

```
long by_value(long x);        // 收一个数
void change(long *p);         // 收地址，通过*p读写对象

long example(long a) {
    long b = a + 2;
    long t = by_value(a);
    change(&b);
    return t + b;
}
```

在不内联、确实发生这些调用的课堂模型中：a只需按值传入；b的地址要交给change，因此通常安排栈槽；t需跨过change这次调用，可放rbx等受保护寄存器，也可存栈。change返回后，b可能被改过，不能继续拿调用前的a+2冒充最新b。

判断流程： 有没有真实地址需求 → 值是否跨调用活跃 → 给定汇编实际写到哪里 → 再谈寄存器是否够。只有C而没有优化前提时，应答“需要/可以/通常”，不要凭源代码声称唯一布局。

7 数组与指针：先确定从哪里开始、一步多大

7.1 一维数组的四种表达式

设`int v[5]={10,20,30,40,50}; int *p=v;`，v起始地址为0x2000，每个int占4字节。

表达式	表示什么	此例结果
p+2	从p开始跨过2个int，得到地址	0x2008
*(p+2)	到刚才的地址取一个int	30
p[2]	与*(p+2)相同	30
&p[2]	取得第2项的地址	0x2008

指针加2不是加2字节，而是加2*sizeof(*p)字节。星号解引用才取内容。(*p)+2则先取10再加2，得到12，和*(p+2)完全不同。

7.2 二维数组：同一个加1，可能跨一行

新例子：

```
int a[3][4] = {
    {10,11,12,13},
    {20,21,22,23},
    {30,31,32,33}
};
int *row = a[1];
```

读图从左往右，一行4个int占16字节，接着下一行。下标从0开始。

```
偏移字节   0   4   8  12 | 16  20  24  28 | 32  36  40  44
数值      10  11  12  13 | 20  21  22  23 | 30  31  32  33
行号            0      |        1       |        2
```

表达式	如何走	结果
a+1	a在此退化为“指向一整行的指针”，跨过1行	第1行的地址，比起点加16字节
a[1]+2	先到第1行，再跨2个int	a[1][2]的地址
*(a[1]+2)	取上面地址的int	22
((a+1)+2)	先取出第1行这一层，再定位行内第2项并取值	22
row[3]	row的起点是第1行，取该行第3项	23

((a+i)+j)等价于a[i][j]。两层星号对应两层数组；*(a+i)得到第i行这个数组，在后面的加法中退化为指向该行首元素的指针，不是凭空多读一次内存中的“行指针”。真正的连续二维数组没有单独存一张行指针表。

7.3 从下标到字节地址，再到汇编

每行C列、每元素s字节时：

$$\mathrm{addr}(a[i][j]) = \mathrm{base}(a) + (iC + j)s$$

例如a[2][1]在线性第$2 \times 4 + 1 = 9$项，偏移$9 \times 4 = 36$字节，值31。注意9是**元素编号**，36才是字节偏移。

机器寻址写法d(B,I,s)计算d+B+I*s。这里s是比例，只能1、2、4、8；它不自动知道C类型。若每行20字节，应先用算术得到i×20，不能直接写比例20。

如果题目给线性编号k，就用i=k/C（整数商）、j=k%C（余数）找行列。如果指针从中间某一行开始，先写清该起点，再加偏移。

连续二维数组的机器布局确实紧接排列；但严格C指针算术还受所指数组对象边界约束。做题可以按明确给定的线性机器模型计算，不把跨子数组越界访问推广成可移植C写法。

8 结构体：成员不能随便紧挨着，整体也要补齐

8.1 对齐就是“这个成员从几的倍数地址开始”

在本讲ABI前提下，char、short、int、long/double的常用对齐分别为1、2、4、8字节。先把结构体起点看作偏移0；其起始地址本身满足整体对齐后，成员偏移也需满足各自对齐。

```
struct Item {
    char tag;
    int count;
    short code;
};
```

顺序	操作	占用偏移	下一个空闲偏移
1	char只需1字节对齐	tag：0	1
2	int要从4的倍数开始，补3字节	填充：1至3；count：4至7	8
3	8已是2的倍数，放short	code：8至9	10
4	整体最大对齐是4，将大小补到4的倍数	尾部填充：10至11	12

所以sizeof是12，不是1+4+2=7。尾部也要补，是因为`Item arr[2]`的下一项从“起点+sizeof(Item)”开始；若大小只有10，第二个元素内部的int就可能错位。

8.2 所有布局题都用同一个步骤

记录“下一个空闲偏移”p。放一个大小s、对齐a的成员时：先把p向上取整到a的倍数，作为成员起点；再加s得到新的p。全部成员放完，按最大成员对齐A再补整体大小。

$$\mathrm{align}(p,a)=a\left\lceil\frac{p}{a}\right\rceil$$

已经是倍数就不再补。例如align(8,4)=8；align(9,4)=12。数组总大小是元素数×元素大小，对齐取元素的对齐，不是数组总长度。

8.3 从汇编偏移反推数组长度：先列候选，不要直接相等

另举一个与原卷数字不同的完整示范：

```
struct P { char a[B]; int t; short s[A]; long u; };
struct Q { int m[A][B]; long y; };
```

已知汇编中P.t偏移4，P.u偏移16，Q.y偏移48。求A、B时，偏移可能包含填充，因此不能直接写B=4、4AB=48。

1. `align(B,4)=4`，得到$0<B\le 4$，B可为1、2、3、4。
2. t从4开始占4字节，s从8开始占2A字节。`align(8+2A,8)=16`，得到$8<8+2A\le 16$，A可为1、2、3、4。
3. `align(4AB,8)=48`，得到$40<4AB\le 48$，所以AB只能为11或12。
4. 在A、B均为1至4的候选中，11无法满足，12可由(3,4)或(4,3)得到。**现有信息无法唯一确定A、B**，需要另一条约束；不该为了凑唯一答案擅自选一个。

这个新例子专门训练“偏移给的是区间约束”以及“多解就保留多解”。若另知s数组占8字节，则2A=8，A=4，再由AB=12得到B=3。代回：a占0至2，补1字节；t占4至7；s占8至15；u从16开始。Q的矩阵占48字节，y恰从48开始。最后必须代回完整布局验证，而非只看某个乘积恰好对。

9 浮点机器代码：先分清地址、整数值和浮点值

9.1 为什么同一个函数会同时用rdi和xmm0

寄存器并非都按同一条队列分参数。整数/地址走通用寄存器，浮点值走XMM寄存器。先看C参数类型，再标入口位置；不能看到函数“第三个参数”就直接填rdx。

新例子：

```
double combine(int *p, float t, double *q) {
    return *p + (double)t + *q;
}
```

p是第1个指针，用rdi；t是第1个浮点值，用xmm0；q是第2个指针，用rsi。假设*p=6、t=1.5f、*q=2.25，最终结果应为9.75。

```
movl     (%rdi),%eax       # 从p指向的内存读int 6
cvtsi2sdl %eax,%xmm1       # 把整数6转换为double 6.0
cvtss2sd %xmm0,%xmm0      # 把float 1.5转换为double 1.5
addsd    %xmm1,%xmm0      # 1.5 + 6.0 = 7.5
addsd    (%rsi),%xmm0     # 读*q，再加2.25，得到9.75
ret                       # 返回double，结果在xmm0
```

9.2 搬位模式、转数值、算浮点，分三类读

类别	例子	做什么	容易混淆的地方
搬运	movss、movsd	复制float/double所用的位模式	不会把整数6自动变成浮点6.0
数值转换	cvtsi2sdl、cvtss2sd	根据原类型的值，重新产生目标类型表示	指令名中si是有符号整数，不是rsi寄存器
浮点运算	addss、addsd	按单/双精度做标量浮点加法	ss/sd不等于整数的s扩展后缀

ss=scalar single（标量单精度），sd=scalar double（标量双精度）。XMM寄存器有128位，但这些标量运算主要处理低32位或低64位的一个数；不能看到xmm就说这次运算处理128位整数。

若把整数位模式`0x00000006`直接按float解释，它并不是6.0；float的6.0编码是`0x40C00000`。所以mov和cvt承担不同任务。题目问“参数q在哪”，答入口rsi；问“*q加载到哪”，则根据实际指令回答，两者不是同一个问题。

10 联合、字节序与防护：从具体内存格子出发

10.1 union的几个成员，入口相同

```
union U {
    unsigned char b[4];
    unsigned int w;
};
```

在当前前提下，b与w都从偏移0开始，共享同一片4字节；不是各占4字节后合计8字节。把b逐字节写成12、34、56、78（均为十六进制）后：

地址从低到高	B+0	B+1	B+2	B+3
实际字节	12	34	56	78
b的下标	b[0]	b[1]	b[2]	b[3]

读取w的规则	最低地址那个12的地位	w的数值
大端	最高有效字节	0x12345678
小端	最低有效字节	0x78563412

字节序改变的是“把多个字节拼成一个数”的顺序，不改变b[0]位于最低地址这件事，也不反转每个字节内部的位。若再写b[0]=AA，小端w变为0x785634AA；w没有单独留着一份旧数据。

10.2 联合大小与数值转换

union大小先取最大成员的大小，再向最大成员对齐的倍数补齐。例如`union { char c[9]; double d; }`最大成员9字节，最大对齐8，因此整体大小16。

用另一成员读取同一片字节属于**表示重解释**。它不是C强制数值转换：`(int)1.5`要求数值1，而把1.5的浮点编码当整数读，是另一件事。本讲按题给ABI、字节序与可表示位模式计算；不把这种读法推广到所有C类型、所有平台都安全相同，某些表示可能无有效值或存在陷阱表示。

读输出题分四步：列低地址到高地址字节 → 按目标成员的宽度分组 → 按字节序拼数 → 按printf格式输出。`%x`用小写十六进制，`%X`用大写；是否有0x、逗号、空格和换行由格式串决定。

10.3 防护机制先分清“阻止什么”

用教学栈图想象一次越界写，地址从左到右升高。具体布局由题目给出，不能保证真实编译器都长这样：

```
[局部字符数组][可能的填充][canary（若启用）][保存的寄存器][返回地址]
        写入超长内容 ──────────────────────────────────────→
```

方法	通俗解释	介入时刻	不能保证什么
正确的长度检查/有界输入	写入前确认目的地装得下	写入之前	长度参数传错仍会出问题；还要给结束零留位置
栈地址随机化	每次运行不总把栈摆在同一地址	进程建立/装载时	不阻止越界写，也不改变同一布局中两个对象的相对偏移
NX/不可执行页	这页允许放数据，但不允许从这里取指执行	CPU尝试执行该页时	不阻止把数据写坏；不自动阻止复用已有代码
canary/栈保护值	返回前检查预先保存的保护值是否被改动	受保护函数入口保存、出口核对	不是修复数据；不是每种越界都必定碰到canary

例如char buf[8]装C字符串时，7个普通字符加1个结束零刚好占8字节；8个普通字符还要再写结束零，共9字节，已经越界。因此“只多写一个零”也可能破坏相邻对象。数组写入向下一个元素推进，地址变大；这与分配栈空间时rsp向低地址移动不是一回事。

canary检查“现在是否等于原来的值”，不是看大小。检测到不一致通常终止程序，而不是把返回地址修好再继续。

10.4 题目问改代码、重新编译、改硬件时怎么想

机制（采用课程常见模型）	程序员改C源代码	重新编译程序	需要什么系统/硬件支持
改用有界输入并传正确容量	是	是	普通指令即可，无需特殊新CPU
栈初始地址随机化	通常否	通常否	操作系统支持；此行不泛指所有代码区域的随机化
NX页执行权限	通常否	仅启用该机制通常不要求重编译	OS设置权限、CPU支持执行权限检查；已有支持就不必更换CPU
编译器插入canary	通常无需手改每个函数	是，需要生成保存与检查指令	编译器/运行库支持，使用已有普通CPU指令

“要不要重新编译”和“要不要改源代码”不同：编译器可在你不改C源码的情况下插入保护指令。硬件列也要分“需要该能力”和“现在必须换机器”。课程历史表将NX记为需要硬件支持，不能背成现代机器每次开启NX都要换CPU。

11 汇编命令速查表：命令、含义、读题要点

下面保留旧表89条的首次出现顺序，末尾补充6条本次常见写法。含义列说明做什么，读题要点列专门列限制、隐含操作和易混点。也可打开独立三栏速查表单独查阅或打印。

第1讲首次出现：数据、运算与扩展

命令	含义	读题要点
`01 movl S, D`	把源的32位复制到目的。	写eax等32位寄存器会清对应64位寄存器高32位；只读eax不会清零。
`02 mov（指令族）`	把一个地方的值复制到另一个地方；源值保留。	AT&T通常按“源,目的”写；普通mov不能直接从一处内存复制到另一处内存。
`03 addb S, D`	把源与目的原值相加，把低8位写回目的。	目的既要读又要写；会改条件码。C的char加法通常先提升为int，不能直接照搬addb。
`04 add（指令族）`	目的原值加上源值，结果放回目的。	宽度看b/w/l/q；会改条件码。定宽机器截断不等于C有符号溢出规则。
`05 shl / sal k, D`	把位向左移，右边补0，左边移出的位丢弃；sal与shl效果相同。	q操作64位，计数取低6位；l操作32位，取低5位。可用立即数或cl作计数，不能直接用rbx。
`06 shr k, D`	把位向右移，左边补0。	常用于无符号数。不要与左侧复制符号位的sar混淆。
`07 sar k, D`	把位向右移，左边补原最高位的值。	负数相当于除以2的幂后向下取整；C整数除法向零截断，负数时可能差1。
`08 movb S, D`	复制1字节，即8位。	写al只改rax最低字节，其他位保留；不会自动得到完整的64位0或1。
`09 movw S, D`	复制2字节，即16位。	写ax只改rax低16位，其他位保留。
`10 movq S, D`	复制8字节，即64位。	rdi是寄存器内容；(%rdi)在mov中是该地址处的内存。方向决定读内存还是写内存。

命令	含义	读题要点
11 `subq S, D`	从目的原值中减去源值，写回64位结果。	第二项减第一项；减rsp是在预留栈空间，不是在初始化栈内存。会改标志。
12 `addq S, D`	把源值加到目的原值，写回64位结果。	加rsp通常回收栈空间；原内存字节没有被擦除。会改标志。
13 `imulq S, D`	双操作数形式：把源与目的相乘，将低64位乘积写回目的。	目的也提供一个乘数；这不是单操作数的rdx:rax完整乘积形式。
14 `andq S, D`	逐位做“与”，结果放回目的；掩码1保留原位，0清零。	不是C的逻辑&&；会改标志，CF与OF清零。
15 `orq S, D`	逐位做“或”，结果放回目的；掩码1强制置1，0不改变原位。	不是C的逻辑或；会改标志，CF与OF清零。
16 `xorq S, D`	逐位做“异或”：相同得0，不同得1；结果放回目的。	与自己异或能清零；也会覆盖比较标志。掩码1翻转、0不变。
17 `lea / leaq 地址式, D`	算出括号里那套地址算式，把算出的数放进寄存器。	不读取该地址处的内存，不改标志。也可用来算5×x等算术；括号不一定代表访存。
18 `movz`（零扩展族）	把窄值复制到宽寄存器，新增高位全部补0。	z表示zero；后两个字母是源宽度、目标宽度。用于保留无符号数值。
19 `movs`（符号扩展族）	把窄值复制到宽寄存器，新增高位都复制源最高位。	s表示sign；用于保留补码有符号数值。此处不是字符串传送movs的用法。
20 `movzbl S8, D32`	取8位源，左边补0，形成32位结果。	常跟在setcc后整理成int的0/1；写eax还会清rax高32位。
21 `movsbq S8, D64`	取8位源，按源符号位扩展成64位。	源FD扩展为FFFFFFFFFFFFFFFD，64位读为−3；正数则补0。
22 `movslq S32, D64`	取32位源，按源符号位扩展成64位。	若eax最高位为1，新rax高32位全1；与movl写32位目标造成高位清零不同。
23 `movsbl S8, D32`	取8位源，按源符号位扩展成32位。	先扩到32位，若写eax又清rax高32位；不等于8→64的movsbq。
24 `addl S, D`	把源与目的的低32位相加，写回32位结果。	写eax等会清相应64位寄存器高32位；标志描述32位运算。

第2讲首次出现：比较、条件与标志

命令	含义	读题要点
25 `cmpq S, D`	用64位“目的−源”的结果设置条件码，两个操作数不被改写。	cmp b,a判断a与b；本身不区分有符号、无符号，后续条件指令决定读法。
26 `testq S, D`	用64位“目的按位与源”的结果设置标志，不写回该结果。	testq rax,rax常检查是否为0；CF、OF清零，不等于比较两个数谁大。
27 `testb S, D`	只按8位做与测试并设置标志，操作数不变。	testb al,al仅检查这个字节；不检查eax或rax的其他位。
28 `cmp`（指令族）	做一次减法式比较，只留下条件码。	读条件跳转前，先确定减法顺序和宽度；后面其他算术可能覆盖这些标志。

命令	含义	读题要点
29 setcc D8	条件成立写一个字节1，否则写一个字节0。	cc是条件占位符；所有setcc都写8位，不清目的寄存器其他部分，也不改标志。
30 jcc 标签	条件成立转去标签，否则执行紧随其后的指令。	读取已经存在的标志；不重新比较，不自动保存返回地址。
31 cmovcc S, D	条件成立，将源复制到目的寄存器；不成立保留该宽度的默认值。	先准备候选和标志；无8位或立即数源形式。内存源不保证条件假就不读；32位形式仍清目标高32位。
32 sete D8	相等条件成立时写字节1，否则写0。	看ZF=1；若要完整int布尔值，通常还需movzbl或预先正确清零。
33 je 标签	ZF=1时跳转。	常用于相等或结果为零；jz是同义名字。条件含义取决于谁设置了ZF。
34 cmove S, D	ZF=1时用源值替换目的候选。	不成立保留目标宽度内旧值；目的须为寄存器。32位形式还要考虑高32位清零。
35 sub（指令族）	从目的原值中减去源，写回结果。	会改标志；目的一源，别颠倒。立即数放源端时带$。
36 inc D	把目的加1。	改ZF、SF、OF等但不改CF；不能一概当成对标志完全等同于add $1。
37 dec D	把目的减1。	同样保留CF；若后续判断借位，不能把旧CF误当成这次减1新产生的。
38 sete / setz; je / jz; cmove / cmovz	相等/结果为零：set写0或1，j决定跳不跳，cmov决定换不换值。	条件ZF=1。三者动作不同，但检测同一个条件；以cmp b,a后解释a与b。
39 setne / setnz; jne / jnz; cmovne / cmovnz	不相等/结果非零：分别写布尔值、跳转、换候选。	条件ZF=0。ne和nz同义；不要求检查CF或SF。
40 setl; jl; cmovl	按有符号数判断“小于”。	条件SF≠OF；l是less，不是32位long。只看SF会漏掉溢出修正。
41 setle; jle; cmovle	按有符号数判断“小于等于”。	条件SF≠OF或ZF=1；比小于多了相等的情形。
42 setg; jg; cmovg	按有符号数判断“大于”。	条件SF=OF且ZF=0；既不能小于，也不能相等。
43 setge; jge; cmovge	按有符号数判断“大于等于”。	条件SF=OF；允许ZF=1，不必另要求非零。
44 setb; jb; cmovb	按无符号数判断“小于”。	条件CF=1；减法有借位。b是below，不是byte宽度后缀。

命令	含义	读题要点
45 setbe; jbe; cmovbe	按无符号数判断“小于等于”。	条件CF=1或ZF=1；不要与有符号le混用。
46 seta; ja; cmova	按无符号数判断“大于”。	条件CF=0且ZF=0；a是above。负有符号位模式按无符号读可能很大。
47 setae; jae; cmovae	按无符号数判断“大于等于”。	条件CF=0；与是否有符号溢出OF无关。
48 cmpl S, D	按32位“目的一源”设置标志。	这里l是32位宽度；不改目的，也就没有“写eax清高位”。
49 cmpb S, D	按8位“目的一源”设置标志。	SF看8位结果最高位；CF看无符号借位；OF看有符号溢出。
50 ret	从当前栈顶读返回地址，rsp加8，然后到该地址继续执行。	整数返回值早已准备在rax；ret弹的是代码地址，不是返回值，也不恢复全部寄存器。
51 xor（指令族）	逐位异或，结果写回目的。	同寄存器异或为0；会改标志，不能随便插在cmp与jcc/cmovcc之间。
52 subb S, D	按8位做“目的一源”，只保留低8位。	CF与OF含义不同；8位结果最高位为1不必然说明原数学差为负。
53 testl S, D	用低32位做与测试，只写标志，不改操作数。	testl eax,eax检查低32位是否全0；不会清rax高32位。

第3讲首次出现：跳转表、除法与调用

命令	含义	读题要点
54 jmp 标签 / jmp *操作数	无条件转到另一个代码位置。	直接跳标签；间接跳转有*。jmp *%rax取寄存器目标，jmp *内存式要读取目标地址；不压返回地址。
55 .quad 地址或数值	让汇编器在这里放一个8字节的数据或地址。	属于汇编器指示，不是CPU命令。跳表里存目标地址，不能把它当成计算结果。
56 .section .rodata	让后续内容进入只读数据节。	属于汇编器指示；不是运行时跳转，也不表示CPU在此读一次内存。
57 .align 8	让后续内容从8字节边界开始，必要时插入填充。	这里按GNU x86语境解释；不是运行时把寄存器乘8。
58 cqto	把rax的64位有符号数扩展成rdx:rax的128位数。	rax不变；rdx全0或全1，旧rdx被覆盖。常为idivq准备被除数。
59 idivq 除数	用rdx:rax组成的128位有符号数除以给定除数。	商进rax，余数进rdx；它们执行前后角色不同。商向零截断；除零或商放不下会异常。
60 call f	压入下一条指令地址，再执行函数f。	压的是回来的位置，不是f入口地址；会改变rsp并写栈，不自动保存其他通用寄存器。
61 pop / popq D	读当前栈顶8字节到普通目标，再把rsp加8。	弹出不擦除内存；若目标是rsp等特殊形式，不能机械套普通目标步骤。

命令	含义	读题要点
62 push / pushq S	先把rsp减8，再把源值写入新栈顶。	pushq rbx保存rbx的数值，不读取rbx指向的内存；源为rsp有特殊取值次序。
63 leave	撤销使用rbp作帧指针的常见栈帧。	常见64位布局下相当于mov rbp,rsp再pop rbp；还要ret才返回，不能用于任意栈布局。

第4讲首次出现：标量浮点

命令	含义	读题要点
64 movss S, D	传送一个32位float的位模式。	在XMM/内存之间看源目的判断读写；不把int自动转换成float。
65 movsd S, D	传送一个64位double的位模式。	这里指XMM标量传送，不是同名字符串指令；不是float→double数值转换。
66 addss S, D	把两个标量float相加，结果写回目的。	ss是单精度标量；有舍入。不会像整数add那样设置ZF/SF/CF/OF。
67 addsd S, D	把两个标量double相加，结果写回目的。	sd是双精度标量；有舍入。目的的标量旧值也是加数。
68 cvtss2sd S, D	把float数值转换成double。	有限float值能精确保留；之前已经丢掉的精度不会恢复。
69 cvtsi2sd（整数转double）	把有符号整数数值转换成double。	si代表signed integer，不代表rsi；源32/64位看操作数或完整后缀，64位整数可能舍入。
70 ucomiss S, D	比较两个float，设置ZF、PF、CF。	NaN会产生无序，此时三者均为1；不要直接套整数jl来判断浮点大小。
71 ucomisd S, D	比较两个double，设置ZF、PF、CF。	OF、SF清零；若可能有NaN，需检查PF区分无序和相等等情况。
72 cvtsi2sdl S32, D	把32位有符号整数转换成double。	末尾l指整数源32位；不是条件less，也不是目标double只占32位。
73 jp 标签	PF=1时跳转。	紧跟ucomis时常检查NaN无序；跟在其他指令后，PF含义要按该指令判断。

第5讲首次出现：具体清零写法

命令	含义	读题要点
74 xorl S, D	做32位异或，把结果写回目的。	xorl eax,eax让整个rax为0，同时改变条件码。

往年题精选首次出现的写法

命令	含义	读题要点
75 leal 地址式, D32	计算地址算式，把结果低32位放进目的寄存器。	不读该地址的内存，不改标志；写32位目标会清对应高32位。
76 jc 标签	CF=1时跳转。	与jb检查同一标志，但叫carry提醒你考虑前条指令是加法进位还是减法借位。
77 incw D16	把低16位加1，结果写回这16位。	保留CF；不清目的寄存器其他高位。

命令	含义	读题要点
78 shlw k, D16	按16位左移，右边补0。	等价salw；传统移位计数用低5位，不是低4位。多位移位的OF不能随便认定为0。
79 subw S, D	按16位做目的−源，写回低16位。	改标志；写16位目标不清其余高位。
80 xorw S, D	按16位异或并写回。	只改目的低16位；会改条件码。
81 testw S, D	按16位做与测试，结果不写回。	CF、OF清零；其他相关标志依16位测试结果设置。
82 andw S, D	按16位做与，写回目的。	和testw的区别在于确实改写目的值；可用于掩码清位。
83 movabsq 立即数, R64	本表形式把完整64位立即数放入64位寄存器。	用于普通立即数编码装不下等情形；movabs还存在绝对地址形式，应看完整操作数。
84 subl S, D	按32位做目的−源，写回结果。	改标志；写eax等会清对应64位寄存器高32位。
85 shrq k, D64	按64位逻辑右移，左边补0。	计数用低6位；65等效移1只适用于机器指令，不能据此认定C移65位合法。
86 jmpq *操作数	64位间接无条件跳转。	*表示目标来自操作数；可来自寄存器或内存。不会压返回地址。
87 and（指令族）	逐位做与，结果写回目的。	与test不同，它会修改目的；CF与OF清零。
88 retq	64位普通返回，从栈中取返回地址并跳回。	题库retq与本表ret相同；别与恢复rbp的leave混为一条。
89 callq f	64位调用，压入下一条指令地址并转到f。	题库callq与本表call相同；调用前仍要自行满足栈对齐和保存规则。

本次补强新增的具体写法

命令	含义	读题要点
90 salq $k, D64 / salq %cl, D64	把64位目的左移，空出的低位补0。	q说明数据宽度64位；次数取低6位，不是只移动6位数据。salq %rbx,%rax不是合法的普通x86写法。
91 shlq $k, D64 / shlq %cl, D64	同salq，是64位左移的另一个名字。	sal和shl没有“左边补符号位”的区别；需要复制符号位的是算术右移sar。
92 sarq $k, D64 / sarq %cl, D64	把64位目的右移，高位填原符号位。	负数右移相当于向下取整；模拟C向零整数除法时，需要按条件加偏置。
93 movzwl S16, D32	把16位源补0扩展成32位。	w是源16位，l是目标32位；写32位寄存器还会清对应高32位。
94 movswq S16, D64	把16位源按符号位扩展成64位。	最高位看源的第15位；不是看扩展后第63位再决定怎么扩展。
95 cvttsd2sil S, D32	把double向零截断成32位有符号整数。	tt指截断；普通有限且截断结果在范围内时，与C转int一致。NaN或超范围时不要硬套普通转换结论。

12 做题时固定问自己的六句话

1. 这里问的是数值、地址，还是位模式?
2. 这一步的类型和宽度是什么？有整数提升或浮点转换吗?

3. 这条指令读了什么，写了什么，是否隐含读写栈或寄存器？
4. 条件来自哪条指令留下的标志？中间有没有覆盖？
5. 这个值跨call后还要不要？谁负责保存？
6. 这条规律依赖哪些前提：ABI、字节序、对齐、浮点精度、题给内存布局？

读完后，可遮住例子右半边，自己补写结果与原因；能解释原因再回做原来卡住的题。尚未收到你的补强后作答，不把本讲已生成当作已经掌握。

13 来源与整理说明

本讲为助手根据你2026–10–11的实际反馈重新编写的专题复习资料。示例中的新数组、结构体与函数是助手补充，不是原卷原题；原题只用于确定应补强的知识范围。

主要依据：本套复习第1至4讲、此前汇编速查表；2025第一次阶段测验带答案版（来源资料未随仓库分发）及已核对的第5讲解析。对应试卷PDF页序：第6题第2至3页，第8题第3页，第11至12题第3至4页，第17题第6至7页，第20至24题第8至10页。这里不是教材印刷页码。

本次保留的必要边界：精度题须检查输入限制；访问计数不等于硬件事务计数；C变量的实际存储受优化影响；二维数组的机器连续布局不替代C边界规则；NX需要硬件能力不等于一律更换CPU。官方手册链接放在相应说明处，便于核对。